PROCEDIMENTO DE CONFIGURAÇÃO DEVSECOPS: INTEGRAÇÃO GITHUB COM SNYK

INTRODUÇÃO

Este guia detalha o procedimento para configurar um ambiente de análise estática de código, como parte da disciplina de Cultura e Práticas DevSecOps, integrando a ferramenta Snyk ao GitHub. Ao final deste procedimento, você terá criado as contas necessárias, configurado um repositório de exemplo e executado sua primeira análise de vulnerabilidades.

IMPORTANTE: O procedimento a seguir considera a interface fornecida pelos fabricantes na data corrente. Em caso de mudanças no processo, por favor, verifique o processo semelhante.

IMPORTANTE: Você deve possuir o Git Bash no Microsoft Windows ou o Git instalado no Linux para conduzir este procedimento. **O Apêndice possui as instruções de instalação**. Também, você pode precisar de baixar e instalar o VSCode para o Windows ou Linux (não é abordada a instalação do VSCode nesta atividade). Ainda, sugere-se realizar este procedimento em uma máquina virtual (esta atividade não aborda a instalação e uso de máquina virtual).

IMPORTANTE: O repositório que será criado deverá ser público.

PROCEDIMENTO TÉCNICO

1. Criação da Conta no GitHub

O GitHub é o SCM (Source Code Management) que será utilizado para este procedimento, sendo a plataforma onde o código-fonte dos projetos de software são armazenados, versionados e gerenciados de forma colaborativa. Para tanto:

1. **Acesse o site do GitHub**
    - Abra seu navegador Web e acesse: https://github.com/join
2. **Preencha os Dados de Cadastro**
    - Você pode realizar SSO (Single Sign-On) com sua conta do Google, por exemplo. Ou,
    - Siga as instruções na tela, informando um nome de usuário, um endereço de e-mail válido e criando uma senha segura.
3. **Verifique seu E-mail**
    - O GitHub enviará um e-mail de verificação para o endereço cadastrado ou utilizado por SSO. Acesse sua caixa de entrada e clique no link de confirmação para ativar sua conta.
4. **Finalize a Configuração**
    - Após a verificação, você pode personalizar seu perfil. Para os fins deste guia, a configuração padrão é suficiente.

2. Criação e Integração da Conta Snyk

O Snyk é a ferramenta utilizada para encontrar, priorizar e ajudar a corrigir vulnerabilidades no código, dependências, contêineres e configurações de infraestrutura.

1. **Acesse o site do Snyk**
    - Em seu navegador da Web, acesse: https://snyk.io/signup
2. **Cadastre-se usando o GitHub**
    - Na página de cadastro, clique na opção **"Continue with GitHub"**. Esta é a maneira recomendada, pois simplifica a integração com o GitHub.
3. **Autorize a Integração**
    - Você será redirecionado para uma página de autorização do GitHub. Clique no botão verde **"Authorize Snyk"**.
    - **Observação:** esta permissão é segura, amplamente utilizada e necessária para que o Snyk possa acessar seus repositórios e realizar as análises de segurança. No próximo passo, será adicionado um repositório para a análise.
4. **Complete seu Cadastro no Snyk**
    - Após a autorização, você retornará ao Snyk. Preencha as informações adicionais solicitadas para finalizar a criação da conta.

Configuração da Integração com o GitHub (Windows e Linux)

Os comandos de configuração são os **mesmos** no Git Bash (Windows) e no Terminal (Linux). **Leia o Apêndice** para realizar a instalação do Git.

1. Configurar Identidade do Git

Essas informações serão anexadas a todos os seus *commits*.

1. **Defina seu Nome de Usuário (Use seu nome real ou o que você usa para commits):**

```
git config --global user.name "Seu Nome Completo"
```

2. **Defina seu E-mail (Use o e-mail associado à sua conta do GitHub):**

```
git config --global user.email "seu_email@exemplo.com"
```

3. **Verifique as configurações:**

```
git config --list
```

2. Autenticação com o GitHub

Para enviar (push) e receber (pull) código do GitHub, o Git requer autenticação. Um método amplamente utilizado, e que será adotado neste laboratório, é o método de autenticação por meio de **Tokens de Acesso Pessoal (PAT)**. Outros incluem, via HTTPS, ou chaves **SSH**.

Tokens de Acesso Pessoal (Person Access Token - PAT)

1. O Git para Windows (Git Bash) vem com o **Git Credential Manager (GCM)**, que facilita a autenticação. Na primeira vez que você tentar interagir com um repositório remoto do GitHub (ex: git push), o GCM abrirá uma janela do navegador solicitando que você faça login no GitHub e autorize o GCM.

2. **No Linux**, o processo de login também pode ser gerenciado pelo GCM (se instalado) ou pode-se gerar um **Token de Acesso Pessoal (PAT)** diretamente no GitHub, que será o processo adotado neste laboratório:

    - Acesse o GitHub, e no canto superior direto da tela, geralmente, no último ícone, faça o seguinte: **Settings > Developer Settings (geralmente, a última opção do painel à esquerda) > Personal access tokens > Tokens (classic)**.
      IMPORTANTE: As configurações da conta são diferentes das configurações do repositório. Para este cenário, você deve estar na página principal da sua conta no GitHub.

    - Gere um novo token, dando a ele o escopo necessário (geralmente, **repo** e **workflow**). Ao clicar em "**Generate new token**", escolha a opção "**Generate new token (classic)**". Na tela a seguir, apresente uma descrição para o token, escolha a data de expiração, marque a opção "**repo**" e a opção **workflow**, para selecionar todos os itens subjacentes. Clique em **Generate token** (final da página). Será gerado um token. Copie o token e o guarde com segurança. **ATENÇÃO**: Nas permissões, você precisa selecionar as opções de permissões para **repo** e para **workflow**. Caso contrário, haverá problemas com

permissão.
IMPORTANTE: Cuidado com este token. Guarde-o em segurança. Este token não estará mais disponível para copiá-lo novamente.

- Ao fazer o primeiro **git push** (log a seguir), quando solicitado pela senha, use o **PAT** que você gerou, em vez da sua senha de conta do GitHub.

IMPORTANTE: Sempre use um arquivo .gitignore em seus projetos para garantir que arquivos desnecessários ou confidenciais (como configurações locais ou dependências) não sejam versionados no SCM. Para saber mais, pesquise sobre como usar o .gitignore.

3. Preparando o Repositório de Teste (java-goof)

Para garantir que a integração esteja funcionando corretamente, utilize o java-goof, um projeto Java intencionalmente vulnerável fornecido pelo Snyk para fins de teste.

IMPORTANTE: Você pode adotar também o Fork do JavaGoof diretamente para a sua conta no GitHub, contudo, neste Laboratório será utilizado o Git para conduzir um processo semelhante.

1. **Crie um Novo Repositório no GitHub**
    - No canto superior direito do GitHub, clique no ícone de **+** e selecione **"New repository"**.
    - **Repository name:** dê um para o projeto (por exemplo, **meu-projeto-java-goof**). O nome deste projeto irá gerar uma URL para o repositório, que será usado posteriormente.
    - **Description:** adicione uma breve descrição (opcional).
    - Selecione **"Public"**. **Importante**, o repositório, para fins deste procedimento, deve ser obrigatoriamente **Public**.
    - Clique em **"Create repository"** (no final da página).
2. **Clone o Código do java-goof para sua Máquina Local**
    - Caso não esteja instalado, instale o Git Bash no Microsoft Windows ou o Git, no Linux, conforme o **Apêndice**.
    - Abra o terminal no Linux (ou Git Bash no Windows).
    - Execute os comandos a seguir para fazer o download do projeto.

O comando '**git clone**' cria uma cópia local de um repositório remoto. Neste caso, estamos copiando o projeto de exemplo vulnerável do Snyk. O endereço eletrônico do projeto Java Goof do Snyk é: https://github.com/snyk-labs/java-goof.

1. No seu terminal (Git Bash ou Linux):
    - Navegue até o diretório onde você quer armazenar o código do projeto (por exemplo, `cd ~/Documents/Projetos`).
    - No Linux, `mkdir Projetos`, e então, `cd Projetos`.

- Use o comando git clone seguido pela URL copiada:

```
git clone https://github.com/snyk-labs/java-goof
```

- O download do projeto Java Goof será realizado para o seu diretório corrente, no caso, Projetos.

Aponte o Repositório Local para o Seu GitHub

Navegue para a pasta do projeto que você acabou de clonar dentro do diretório Projetos:

```
cd java-goof
```

Use o comando abaixo para definir a Branch principal, como Main:

```
git branch -M main
```

Agora, copie a URL do seu repositório criado no GitHub (por exemplo: https://github.com/seu-usuario/meu-projeto-java-goof.git), e execute o comando a seguir para vincular sua cópia local ao seu repositório no GitHub. Substitua URL_DO_SEU_REPOSITORIO pela URL que você copiou. Será adotado o comando **'git remote set-url'**, que altera a URL do repositório remoto chamado 'origin'. 'origin' é o nome padrão do servidor de onde o projeto foi clonado. Com este comando, garante-se que será enviado o código para o seu repositório criado, e não para o original do Snyk.

```
git remote set-url origin URL_DO_SEU_REPOSITORIO
```

Por exemplo:

`git remote set-url origin` https://github.com/seu-usuario/meu-projeto-java-goof.git

Envie o Código Para o Seu Repositório

Execute o comando abaixo para fazer o upload do projeto java-goof para o seu repositório no GitHub.

```
git push -u origin main
```

IMPORTANTE: Após o comando acima, você irá precisar do token (PAT) emitido.

OBSERVAÇÃO: Se o seu Git usar master como branch padrão, o comando pode ser: `git push -u origin master`.

4. Analisando o Repositório com o Snyk

Com o código vulnerável em seu GitHub, o próximo passo é instruir o Snyk a analisá-lo.

1. **Adicione seu Projeto no Snyk**
    - Faça login na sua conta Snyk e acesse o Dashboard.
    - Clique em **Projects** no menu superior e, em seguida, clique no botão **"Add project"**.
2. **Selecione o GitHub**
    - Na lista de integrações disponíveis, clique em **"GitHub"**.
3. **Selecione o Repositório**
    - O Snyk exibirá a lista de repositórios da sua conta do GitHub. Encontre o seu repositório (por exemplo, meu-projeto-java-goof) e marque a caixa de seleção ao lado dele.
    - Clique no botão **"Add selected repositories"** no canto superior direito.
4. **Aguarde a Análise**
    - O Snyk iniciará automaticamente o processo de importação e análise. Ele identificará o arquivo de manifesto do projeto (pom.xml para Java/Maven) e analisará todas as dependências em busca de vulnerabilidades conhecidas.

5. Visualizando Resultados e Realizando um Novo Scan

Após a conclusão da análise, o Snyk apresentará um relatório completo.

1. **Acesse o Relatório de Vulnerabilidades**
    - Na página/guia **"Projects"**, clique no nome do seu repositório.
    - Você verá um painel com o resumo das vulnerabilidades encontradas, classificadas por nível de severidade (Critical, High, Medium, Low).
2. **Explore os Detalhes de uma Vulnerabilidade**
    - Clique em qualquer vulnerabilidade da lista para expandir os detalhes. O Snyk fornecerá informações cruciais, como:
        - **Descrição da falha:** o que é e qual o risco associado.
        - **Pontuação de gravidade (CVSS):** um padrão numérico que indica a gravidade técnica.
        - **Caminho da dependência:** mostra como essa biblioteca vulnerável foi adicionada ao projeto.
        - **Recomendação de correção:** geralmente, sugere a atualização para uma versão mais recente e segura da biblioteca.
3. **Como Realizar um Novo Scan (Re-teste):**
    - O Snyk monitora seu repositório e executa análises automaticamente a cada novo *commit* na branch principal.
    - Para forçar uma nova análise manualmente (por exemplo, após aplicar uma correção):
        - Navegue até a página do Projects dentro do Snyk.
        - Haverá diversos componentes, com /pom.xml (todolist-web-struts/pom.xml, por exemplo). Ao clicar em um destes projetos, abaixo da guia Overview, após a data de criação (Created), haverá o link "**Retest now**".
        - Clique no botão **"Retest now"**.
    - Esta ação é útil para validar imediatamente se as correções aplicadas resolveram as vulnerabilidades reportadas.

CONCLUSÃO

Você completou o procedimento de configuração da integração entre o GitHub e o Snyk e realização de análise estática de código.

APÊNDICE: INSTALAÇÃO DO GIT

Instalação e Configuração Git/GitHub

Instalação do Git Bash no Microsoft Windows

O Git Bash é uma aplicação que fornece um ambiente de emulação de terminal Linux/Unix no Windows, permitindo usar comandos Git e muitos outros utilitários Linux (como ssh, scp, cat etc.). Antes de iniciar, instale o VSCode na sua máquina virtual ou no seu computador.

1. Baixar o Instalador do Git Bash

1. Acesse o site oficial do Git: https://git-scm.com/install/windows
2. O download do instalador da versão mais recente para Windows deve começar automaticamente. Se não, clique no link apropriado para a versão de 32 ou 64 bits.

2. Observações e Execução da Instalação

1. Execute o arquivo baixado (geralmente Git-*-64-bit.exe).
2. **Licença:** clique em **Next**.
3. **Local de Instalação:** mantenha o padrão, a menos que tenha um motivo para alterar, e clique em **Next**.
4. **Seleção de Componentes:** mantenha as opções padrão (certifique-se de que o **Git Bash Here** esteja marcado para facilitar o acesso) e clique em **Next**.
5. **Menu Iniciar:** mantenha o nome padrão do menu e clique em **Next**.
6. **Escolhendo o Editor Padrão:** escolha o editor de texto que você prefere (por exemplo, Vim (Linux), VSCode (Linux/Windows)) para ser o padrão do Git e clique em **Next**.
7. **Ajustando o PATH (Importante: opção fundamental):**
    - **Recomendado (Padrão):** marque **"Git from the command line and also from 3rd-party software"**. Esta opção permite usar o Git tanto no Git Bash quanto no Prompt de Comando/PowerShell e aplicativos externos. Clique em **Next**.
8. **Backend SSH:** mantenha a opção padrão (**"Use bundled OpenSSH"**) e clique em **Next**.
9. **Biblioteca SSL:** mantenha a opção padrão (**"Use the OpenSSL library"**) e clique em **Next**.
10. **Configuração de conversão de fim de linha (Line Ending):**
    - Se você for adotar em um ambiente misto (Windows e Linux/macOS), a opção padrão **"Checkout Windows-style, commit Unix-style line endings"** (CRLF para LF) é geralmente a mais segura. Clique em **Next**.

11. **Emulador de Terminal:** selecione **"Use MinTTY (the default terminal of MSys2)"** para ter o Git Bash. Clique em **Next**.
12. **Opções Adicionais:** mantenha as opções padrão (ex: Fast-forward ou merge) e clique em **Next**.
13. **Componentes Experimentais (Opcional):** você pode desmarcar/manter e clicar em **Install**.
14. **Conclusão:** após a instalação, você pode marcar a caixa **"Launch Git Bash"** e clicar em **Finish**.

3. Verificação (Windows)

1. Abra o **Git Bash** (pesquise no Menu Iniciar).
2. Digite o comando para verificar a versão:

```
git --version
```

Se a versão aparecer, a instalação foi bem-sucedida.

Instalação do Git no Linux (Distribuições baseadas em Debian/Ubuntu)

Em muitas distribuições Linux, o Git já vem pré-instalado. Contudo, primeiro, basta verificar.

1. Verificação

1. Abra o terminal.
2. Verifique se o Git já está instalado:

```
git --version
```

Se uma versão for exibida, o Git já está instalado. **Se não**, prossiga para a instalação.

Instalação via Gerenciador de Pacotes

Para distribuições baseadas em Debian/Ubuntu (as mais comuns):

1. Atualize o índice de pacotes:

```
sudo apt update
```

2. Instale o Git:

```
sudo apt install git -y
```

Para distribuições baseadas em Fedora/CentOS/RHEL:

1. Use o dnf ou yum:

```
sudo dnf install git-all -y
```

Ou

```
sudo yum install git-all -y
```

2. **Verificação:** execute novamente `git --version` para confirmar.